

Storehouses



Data Science and Database Technology
Politecnico di Torino

Problem specifications

Design a data warehouse to manage the storehouses of an Italian household appliance company, addressing the following issues. *The company has storehouses all over Italy.* The management needs to analyze the current storehouse usage to decide which ones to expand, reduce, or partially rent.

The management periodically takes out theft insurance against robbery in its storehouses. The cost of the insurance depends on the average daily amount of products stored in the storehouses and on their value. Different insurances exist and the management needs to decide which one is the best. Some insurance policies are specific to a particular storehouse, while others are common to different storehouses of the same city, province or region.

The insurance costs depend on the following daily measures:

- Number of products stored in each storehouse for each type of product
- Value of the products stored in each storehouse for each type of product

Regarding the usage of the storehouses (i.e., surface taken by goods), the management needs to analyze the daily percentage of free surface with respect to the total available surface of each storehouse, according to the storehouse location (city, province, and region). The total available storehouse surface may change, since some areas of the storehouse may be rented or be temporarily unavailable. The analysis will be performed for each day, month, trimester, 4-month period, semester, and year.

To assess the insurance costs, the management needs to analyze the average daily number of products stored in each storehouse and the daily total value of the products for each date, month, 3-month period, 4-month period, semester, and year. This information must be available for each product type, product category, storehouse, and storehouse location (city, province, and region).

The following are some of the frequent queries the management is interested in:

- In the first trimester of 2013, regarding the storehouses in Turin, select the total value of the products stored in each storehouse at any given date, and select the average daily total value of the products in each storehouse during the previous week (including the current date).
- In 2014, for each city and date, select the percentage of daily free surface of the storehouses. Give a rank to the results (rank 1 is the lowest percentage).
- In the first 6 months of 2014, select the percentage of free surface for each storehouse and date.
- In 2013, select the average daily total value of products for each storehouse and month.
- In 2015, select the average daily total value of products for each region.
- In 2014, select the average percentage of daily free surface for each month and region.

Design

- Design the data warehouse needed to manage the needs of the appliance company to meet the requirements described in the problem specification. The designed data warehouse must also be able to efficiently respond to all of the frequent queries proposed in the problem specification.

The data warehouse must contain information for the last 10 years. To properly implement the data warehouse, the following information has been provided:

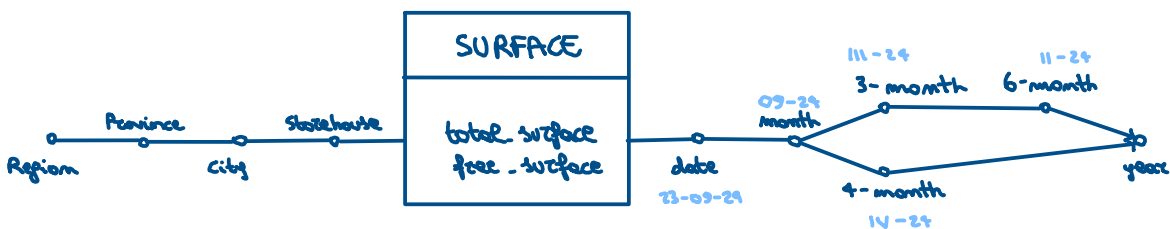
- Product types: ~100
- Product categories: ~10
- Storehouses: ~500
- Storehouses in Turin: ~5
- Cities: ~200
- Regions: ~20
- Provinces: ~90

- Write all the frequent queries of the "problem specifications" using the extended SQL language.

- Define the materialized view that could be used to efficiently answer all the following queries:

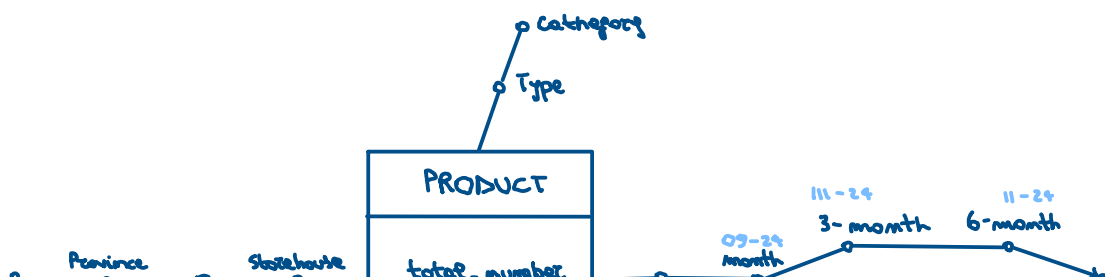
- In the first half of the years 2015 and 2016, considering only the warehouses in the Piedmont region, select the total value of products and the total number of products for each province and month.
- In the year 2013, select the average monthly value of products for each province of the storehouse and semester.
- In the year 2014, select the total value of products for each region and 3-month period.

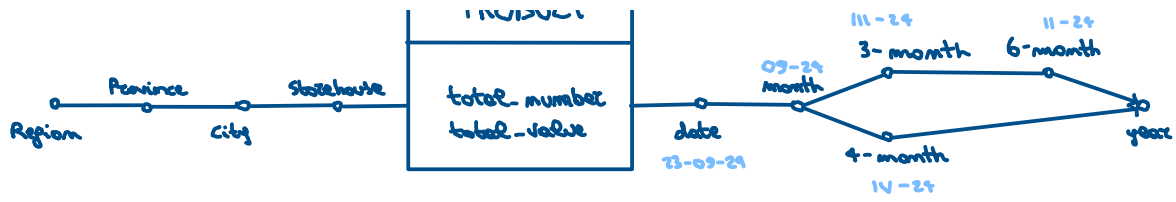
FACT TABLE (1)



STOREHOUSE (SID, Storehouse, city, Province, Region)
TIME (TID, date, month, 3M, 4M, 6M, year)

SURFACE (SID, TID, total_surf, free_surf)





STOREHOUSE (SID, Storehouse, city, Province, Region)
 TIME (TPID, date, month, 3M, 4M, 6M, year)

TYPE (TPID, type, category)
 PRODUCT (SID, ID, TPID, total-number, total-value)

Italian wines



Data Science and Database Technology
Politecnico di Torino

Problem specifications

As part of the enhancement of the excellence of Made in Italy, the Italian Government wants to analyze the export of Italian wines abroad, to understand, for example, which are the most popular wines, from which areas of Italy they come, and which are the main foreign countries importers. The Government has collected from Italian wine companies the list of purchase orders. Each purchase order is characterized by date of the order, amount in euros, quantity in liters, type of wine, type of packaging (e.g., bottle, carton, demijohn, etc.), VAT number of the wine company of origin, foreign country of destination. The analysts, starting from the purchase orders, want to create a data warehouse which synthesizes the collected data in order to answer a set of queries. During the implementation phase, analysts integrate order data with context data in their possession, necessary to carry out the analysis of interest. In particular, each type of wine is associated to the list of quality certifications (DOC, DOP, or DOCG) it owns. For example: the wine type "Barolo" is always associated to the DOCG certification, the wine type "Nebbiolo" is always associated to DOP and DOCG certifications, etc. Each certification can be associated to more than one type of wine. Some types of wine can be without certifications. Each order is also associated to its size: orders up to 100 liters are considered small, medium the ones up to 1000 liters, and large the ones above. Finally, for every winery, it is possible to find out from the VAT number the complete personal data (name, address, province, region, geographical area of the region).

The queries analysts are interested in concern the total quantity of wine exported (in liters), and the average price per liter, according to the variation of:

- month, 2 months period, 3 months period, semester, and year
- province, region and geographical area (North, Center, South) of the winery
- wine type
- quality certifications
- packaging type
- foreign country of destination, and its continent
- order size (small, medium, large)

The data warehouse will track information for the years 2010 through 2013. Following are some specific queries:

- Considering only wines that have "DOC" certification exported to Asia, select the average price per liter for each year, the percentage of liters exported in the year compared to the total for all years, and the cumulative yearly total of liters exported. Perform the analysis separately for each type of packaging.

- For each region, select the average price per liter, the average number of liters of wine exported per province, and the percentage of liters of wine exported from each region with respect to the total of the geographical area to which it belongs (North, Center, South). Perform the analysis separately for each year.
- Assign a rank to each foreign country according to the total quantity of wine imported. Perform the analysis separately for each type of wine. Consider only large orders.

Design

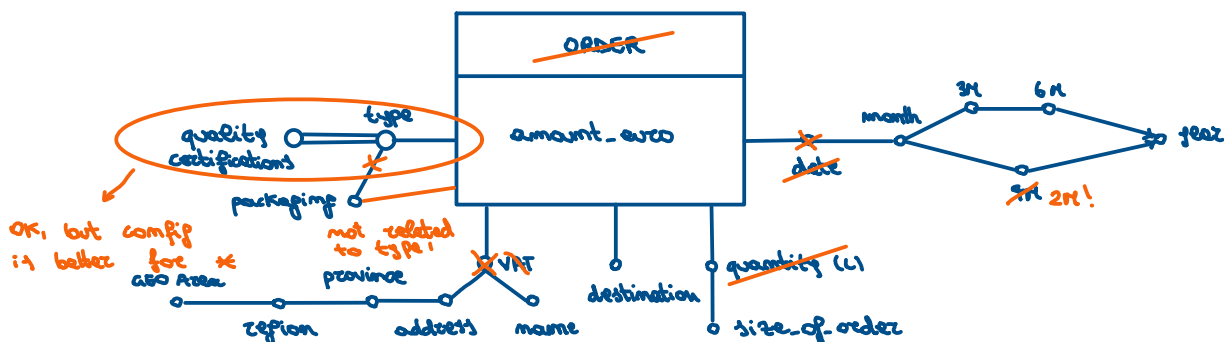
- Design the data warehouse needed to manage the needs of the appliance company to meet the requirements described in the problem specification. The designed data warehouse must also be able to efficiently respond to all of the frequent queries proposed in the problem specification.
- Write frequent queries (a) and (b) using the extended SQL language.

double? * I know exactly the values, there are many => CONFIA BETTER

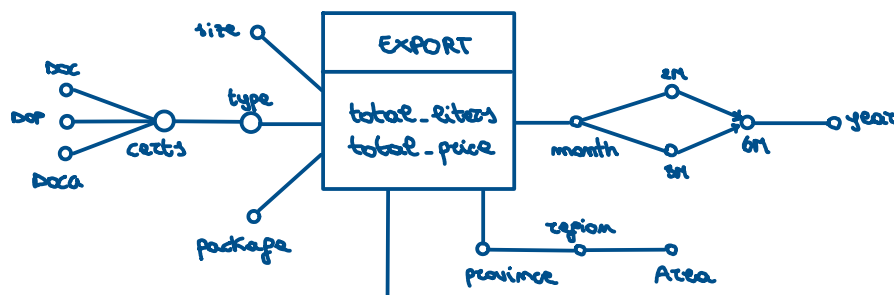
Note: sizes are 'possible' -> they can change, to no config

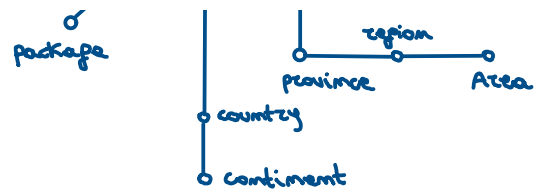
the analysis does not require the 'date'!
VAT is not necessary!

Mine



Solution





International parcels



Data Science And Database Technology
Politecnico di Torino - School of Information Engineering
Master of Science in Computer Engineering

Design a data warehouse to analyze the business of an international parcels service with more than 200 branches all over the world, addressing the following issues.

Problem specifications
The branches manage dispatches of different products for many third party companies. The current information system is heterogeneous, thus each branch has its own data for its own business activities.

Each dispatch is handled by a single branch. The dispatch consists of one or more routes and every route has a **departure** and a **destination** place (city, province, region, and state). The management needs to analyze the global flows of delivered goods to decide which branches to expand or reduce and to take strategic business decisions.

The analysis of the good flows is performed taking into account the income, the weight (expressed in kg), and the volume of the delivered goods. In particular, the management is interested in analyzing the income of each branch and in comparing it to the income of each district to which the branch belongs. A further analysis is performed on the profitability for **different categories of goods** and for **different carrier types** (air, rail, sea freight, etc).

To decide which branches to expand or reduce, the management needs to analyze the shipping routes in terms of income, volume, and weight of the delivered goods.

Facts
Eventually, the management needs to analyze the average income, the average weight, and the average volume of goods delivered in different years, semesters, 4-month periods, trimesters, 2-month periods, months, days of the month and days of the week.

The following are some of the frequent queries the management is interested in:

- Considering only Italian routes, for each category of goods and for each year, select the average daily income for each month and the total monthly income since the beginning of the year.
- Considering only air carriers, select the yearly average income per unit of volume for each destination province, and the percentage of such income compared to the yearly average income per unit of volume of the destination state.
- In 2006 for each route, in terms of departure and destination region, select the monthly average income per unit of weight (in kg) of the delivered goods, and the average daily income for each month of the goods delivered on that route.
- For each branch district and carrier type, select the total income for each month and the total volume of goods delivered in each month. Rank the results according to the total monthly volume (the highest is 1°).

Page 1 of 2

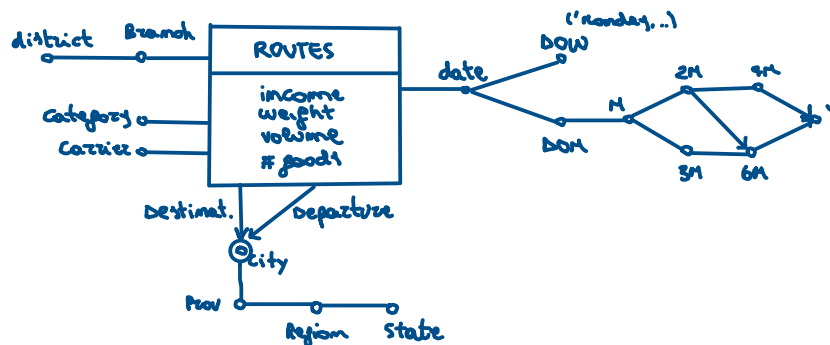
- In 2005, for each route, in terms of departure and destination city, select the yearly average income per unit of weight of the delivered goods, and the average daily income for each considered route.
- In 2005 and 2006, considering only shipping by sea, select the half-year average income per unit of volume for each branch and for each departure region, and the half-year average income per unit of weight.
- For each departure city and for each branch district, select the 4-month period income and the total volume of delivered goods for each 4-month period.

Design
The data warehouse will store information of 2004, 2005, 2006 and 2007. The following cardinalities are known (suppose data is uniformly distributed):

- Good categories: ~20
- Branches: ~200
- Branch districts: ~50
- Different carrier types: ~5
- Cities: ~1000
- Provinces: ~200
- Regions: ~100
- States: ~20

- Design the data warehouse to address the described issues. In particular, the designed data warehouse must allow efficient execution of all the queries described in the specifications.
- Write the frequent queries (a), (c) and (d) of the "problem specifications" using the extended SQL language.
- Considering the designed data warehouse and its cardinalities, decide whether and which materialized views are convenient to improve response time of the frequent queries (consider all the frequent queries). Explain reasons for your choices.

Page 2 of 2



TIME (TID, date, DOW, DOM, M, 2M, 3M, 4M, 6M, Year)

BRANCH (BID, Branch, district)

LOCATION (LID, City, Prov, Region, State)

ROUTES (TID, BID, Category, Carrier, Departure LID, Destination LID, totIncome, totVolume, totWeight, #g)

Schema concettuale

Gestione dei rami che hanno più valori possibili

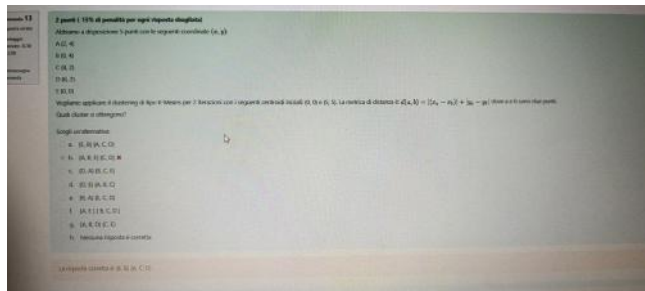
Usiamo il doppio collegamento (=) quando abbiamo più possibilità e non sono tutte note (troviamo nel testo "ad esempio telefono, tablet, pc)

Usiamo il collegamento singolo (-) quando invece tutte le opzioni possibili sono elencate (telefono o tablet o pc)

Usiamo i multi collegamenti (più rami che escono dal nodo) quando più di un'opzione è plausibile (un'aula può contenere uno o più tra i seguenti elementi: proiettore, LIM, armadietto, registro cartaceo)

Clustering

K-means : compute centroids



1)

	C_1	C_2
A	6	4
B	1	6
C	6	4
D	8	4
E	0	10

$$C_1 = \frac{B_1 + E_1}{2}, \frac{B_2 + E_2}{2} \Rightarrow \frac{0+0}{2}, \frac{4+0}{2} \Rightarrow C_1 = (0, 2)$$

$$C_2 = \frac{A_1 + C_1 + D_1}{3}, \frac{A_2 + C_2 + D_2}{3} \Rightarrow \frac{2+4+6}{3}, \frac{4+2+2}{3} \Rightarrow C_2 = (4, 2.67)$$

2)

	C_1	C_2
A	4	3.33
B	6	10.67
C	8	0.67
D	10	6.67
E	2	6.67

$\Rightarrow$ Clusters: $C_1: \{B, E\}$, $C_2: \{A, C, D\}$

DBSCAN

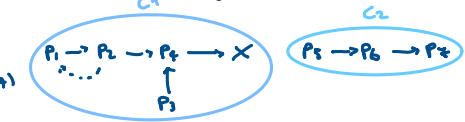


distances:

	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10
P1	-	1	6	4	7	9	10			
P2	1	-	4	2	5	7	8			
P3	6	4	-	2	3	3	4			
P4	4	2	2	-	3	5	6			
P5	7	5	3	3	-	2	3			
P6	9	7	3	5	2	-	1			
P7	10	8	4	6	3	1	-			

$\{P1, P2, P3, P4\}, \{P5, P6, P7\}$

Note: $d \in E$ "appropia"



Core point: has more than MinPts points within the circumference of radius Eps

Border point: has less than MinPts within the circumference of radius Eps, but it is in the vicinity of a core point

Noise point: none of the previous

MAX LINKAGE

	a	b	c	d	e					
a	/	7	3	11	11	{a,c}	/	7	11	11
b	7	/	6	6	6	b	7	/	6	6
c	3	6	/	10	11	{a,c}	11	6	/	
d	11	6	10	/	4	d	11	6	4	/
e	11	6	11	4	/	e	11	6	4	/

After two iterations we have {a,c}, {b}, {d,e}
 After three iterations we have {a,c}, {b,d,e}

Association

2 punti (penalità 15% per risposta errata)

Viene data la base dati transazionale mostrata qui sotto. Si applichi l'algoritmo Apriori per l'estrazione di itemset frequenti.

Transactions	A	B	C	D	E	
0	ACD	5	6	8	3	6 → CBEAD
1	BC					
2	BE					
3	ABCE					
4	AC					
5	AC					
6	ABE					
7	BCDE					
8	CDE					
9	BCE					

Itemset $L_2 = \{CA\}$
Itemset $L_3 = \{\}$

Nota: probabilmente non serve riordinare!

Il valore di minsup è 2 (un itemset è frequente se appare in almeno 2 transazioni).

Quali sono gli itemset candidati di lunghezza 3 che vengono generati nel join step e poi prunati nel prune step?

Si utilizzi l'ordinamento alfabetico per stabilire un ordine fra gli items.

1.3 punti (penalità 15% per risposta sbagliata)

Si vuole applicare l'algoritmo Apriori per l'estrazione degli itemset frequenti con minsup=4. Un itemset è frequente se compare in almeno 4 transazioni. La tabella seguente contiene gli itemset candidati di lunghezza 2.

itemset	support	L_2
{A, B}	7 > 4	✓
{A, C}	6 > 4	✓
{A, D}	8 > 4	✓
{B, C}	1 < 4	×
{B, D}	2 < 4	×
{C, D}	3 < 4	×

Dopo il prune step effettuato applicando il principio Apriori, quanti sono i candidati di lunghezza 3 selezionati per il passaggio del supporto nel dataset?

Scegli un'alternativa:

a. 1
b. 5
c. 0
d. 2
e. 3
f. 4

Risposta errata.
La risposta corretta è 0.

L_2	L_3	C_3
{A, B}	7	{A, B, C} {A, B, D} {A, B, E}
{A, C}	6	{A, C, B} {A, C, D} {A, C, E}
{A, D}	8	{A, D, B} {A, D, C} {A, D, E}

↳ SE PER ESEMPIO $\{C, D\} = 1$, ALLORA
POTEVO GENERARE $\{A, C, D\}$ E SAREBBE STATO VALIDO

Note:

1. Apriori sembra NON RICHIEDERE L'ORDINAMENTO!
2. * When generating C_k , you have to "watch" only in L_{k-1} to see possible combinations!

FP-Tree

Important: everytime you compute a CPB you have to write the header table (# elements), sort it and rewrite the CPB with that order!!!

FP-Tree: how to

1. Devi costruire prima l'albero iniziale, ordinando i supporti
2. Generi i percorsi dal primo punto X-CPB
3. Successivamente, ordini l'header table di questo percorso e riordini il CPB
 - a. Se è richiesto, ripeti di nuovo il passaggio per generare i percorsi di un eventuale XY-CPB
4. A questo punto verifichi quali tra quei percorsi rispecchiano il vincolo di minsup (costruisci l'header table e verifichi che ogni elemento di un percorso abbia supporto ≥ 2)

Esempio con minsup = 2:

Se generi AC, A, B come percorsi disponibili alla fine

E hai come header table A=4, B=1 C=2

Puoi accettare AC ed A, non B in quanto il suo supporto (1) è minore del minsup (2)

Queries and Trigger (notes are in Italian)

SQL

- Quando inserisco una **GB** "modifico" la **struttura** della tabella, andando a iterare su ogni tupla composta dagli attributi **della GB**.
 - **GB Mese, Comune** implica che ogni operazione successiva avvenga sulla tupla mese-comune.
 - **Es.** mese1-comune1, mese1-comune2, mese2-comune1, mese2-comune2, ...
- Dunque fare **SUM(ingressi)** significa calcolarli partendo da **quelle tuple mese-comune** (somma gli ingressi di ogni tupla mese-comune)
 - O ancora meglio, fare una **partition** implica fare dei calcoli partendo da quelle tuple, senza modificare la struttura della tabella (facendo GB si riduce il numero di righe) generando un risultato del tipo:
 - mese-comune + risultato partition (**Es:** TotaleIngressi)
- Nello specifico, a seguito di un'alterazione dovuta a GB NomePiscina, mese, comune:
 - **MAX(SUM(numeroIngressi) OVER (PARTITION BY Mese, Comune))**
Genera, per ogni piscina (resta fuori da partition!) il valore più alto della somma totale degli ingressi mensili tra tutte le piscine dello stesso comune nello stesso mese
 - **Es:** Einaudi-Torino-Giugno → MAX(...). Nello specifico, la query genera una tabella del tipo:

Piscina1	Torino	Giugno	numIngressiMensili
Piscina2	Torino	Luglio	numIngressiMensili
Piscina3	...	...	...

A cui poi viene applicato **MAX** per poi ritornare il valore più alto di ingressi tra le piscine dello stesso comune nello stesso mese (Supponiamo che Torino abbia 6 piscine: per ogni piscina, ogni mese, troveremo il numero di ingressi più alto registrato in quel mese da una delle 6 piscine)
- Come gestire 'lasso di tempo' cumulativo allo scorrere di 'finestra di tempo' (**annual** income over the **3M**)
Es. Stipendio annuale cumulativo dello scorrere dei trimestri:
SUM(SUM(Incasso)) OVER (PARTITION BY Anno ORDER BY 3M ROWS UNBOUNDED PRECEDING)
 - **PARTITION BY** 'attributi GB comuni' + 'lasso di tempo' (**Es:** PARTITION BY Region, Anno)
 - **ORDER BY** 'finestra di tempo' (3M)

TRIGGERS and Materialized View

Query d'interesse e definizione della vista materializzata: in questa sezione bisogna scrivere le query di interesse e successivamente riportare tutti i loro elementi in una query per inserirli nella vista materializzata.

Solitamente sono molto semplici, ma a volte tirano fuori comunque roba da bestemmiare. Non è raro che ci sia una somma cumulativa qui. Esempio:

1)
 SELECT SUM(#Visite), SUM(OreTotali)/SUM(#Pazienti), SUM(#Pazienti)/COUNT (DISTINCT
 Provincia)
 FROM APS, TRIAGE, OSPEDALE TEMPO
 WHERE colore = 'giallo'
 GB Regione, Anno

2)
 SELECT SUM(SUM(#Pazienti) OVER (PARTITION BY Anno ORDER BY Mese ROWS UNBOUNDED
 PRECEDING))
 FROM APS, TEMPO, TRIAGE
 WHERE join AND CodColore = 'Rosso' AND ModalitaArrivo = 'ambulanza'
 GB Mese, Anno

3)
 SELECT SUM(#Pazienti), SUM(#Visite)/SUM(#Pazienti)
 FROM OSPEDALE, TEMPO, APS
 WHERE join AND Regione='Piemonte' AND Ortopedia=1 AND Geriatria=0
 GB Provincia, 6-Mesi

Blocco A

SELECT Mese, 6-Mese, Anno, Provincia, Regione, CodColore, ModalitaArrivo, Ortopedia,
 Geriatria, SUM(#Visite) As TotVisite, SUM(#Pazienti) TotPazienti, SUM(OreTotali) As TotOre
 FROM OSPEDALE O, TRIAGE TR, TEMPO T, ACCESSI-PRONTO-SOCCORSO APS
 WHERE APS.CodO = O.CodO AND TR.CodTriage = APS.CodTriage AND T.CodT = APS.CodT
 GB Mese, 6-Mesi, Anno, Provincia, Regione, ModArrivo, CodColore, Ortopedia, Geriatria

Insieme minimale: bisogna ridurre al minimo gli elementi per definire le righe di tale vista materializzata. Elementi come 3M, 6M, Year per esempio possono tutti essere definiti da Mesi; elementi come Regione, Stato invece possono essere definiti a partire da Provincia. Esempio:

Insieme minimale: Mese, Provincia, ModArrivo, CodColore, Ortopedia, Geriatria

Definizione del trigger. Qui si definisce il trigger, facendo in ordine le seguenti cose:

- a. Dichiarazione delle variabili (elementi statici della MV, quindi non quelli "nuovi" che arrivano dall'inserimento nelle tabelle) dopo aver scritto la sintassi del trigger (*rivedila che tu sei stronzo e sicuro te la scordi, già ti vedo a bestemmiare dopo la correzione per aver perso punti importanti qui*)
- b. A seguito di BEGIN, dobbiamo riempire tutte le variabili dichiarate pescando dalle tabelle che contengono quei dati, mettendo semplicemente CodTabella = :New.CodTabella
 Pattern comune a tutti i trigger è calcolare COUNT(*) INTO N per vedere se la tabella presenta già quei dati (UPDATE) oppure sono da inserire (INSERT)
 - c. Definire l'IF che controlla N>0: nel ramo true gestire l'update, nel ramo false gestire l'insert

Esempio:

a.
 CREATE OR REPLACE TRIGGER InsertViewAccessi
 AFTER INSERT ON ACCESSI-PRONTO-SOCCORSO
 FOR EACH ROW
 DECLARE
 me, 6m, an DATE;

```
prov, reg, col, mod varchar(10);  
ort, ger BOOLEAN;  
N INTEGER;
```

b.
BEGIN

```
SELECT Mese, 6-Mesi, Anno INTO me, 6m, an  
FROM TEMPO  
WHERE CodT = :NEW.CodT
```

```
SELECT Provincia, Regione, Ortopedia, Geriatria INTO prov, reg, ort, ger  
FROM OSPEDALE  
WHERE CodO = :NEW.CodO
```

```
SELECT ModArrivo, CodColore INTO mod, col  
FROM TRIAGE  
WHERE CodTriage = :NEW.CodTriage
```

```
SELECT COUNT(*) INTO N  
FROM ViewAccessi  
WHERE Mese = me, Provincia = prov, ModArrivo = mod, CodColore = col, Ortopedia = ort,  
Geriatria = ger
```

```
c.  
IF (N>0) THEN  
    UPDATE ViewAccessi(  
        SET #Visite += :NEW.TotVisite, TotPazienti += :NEW.TotPazienti, TotOre  
        += :NEW.TotOre  
        WHERE Mese = me, Provincia = prov, ModArrivo = mod, CodColore = col, Ortopedia  
        = ort, Geriatria = ger  
    ELSE  
        INSERT INTO ViewAccessi(Mese, 6-Mese, Anno, Provincia, Regione, CodColore,  
        ModalitaArrivo, Ortopedia, Geriatria, TotVisite, TotPazienti, TotOre)  
        VALUES (me, 6m, an, prov, reg, col, mod, ort,  
        ger, :NEW.TotVisite, :NEW.TotPazienti, :NEW.TotOre)  
    ENDIF;
```

END;

View Equivalence and Conflict Serializability

View Equivalence

Two schedule are equivalent if they have same:

- reads-from: $ri(x)$ reads-from $wj(x)$ if $wj(x)$ precedes $ri(x)$ with $i \neq j$ and there is no $wk(x)$ between them
- final write: $w_i(x)$ is a final write if it is the last write of x in the schedule
 - $S1 = w0(x) r2(x) r1(x) w2(x) w2(z)$ and $S2 = w0(x) r1(x) r2(x) w2(x) w2(z)$ are view equivalent
 - $S1 = w0(x) r2(x) r1(x) w2(x) w2(z)$ and $S3 = w0(x) r1(x) w2(x) r2(x) w2(z)$ are NOT view equivalent (**no reads-from**)
 - $S1 = w0(x) r2(x) r1(x) w2(x) w2(z)$ and $S4 = w2(x) w0(x) r1(x) r2(x) w2(z)$ are NOT view equivalent (**no final write**)

Conflict Serializability

Let's define conflicts: $Wi \rightarrow Wj$, $Wi \rightarrow Rj$, $Ri \rightarrow Wj$ with $i \neq j$

- Very important is that $Ri \rightarrow Rj$ is **NOT A CONFLICT**

After that, we should first check if the schedule is serializable: we draw the conflict graph and we check there's no cycle!

Moreover, to check equivalent schedules they should respect the order of transactions

- Es: Let's suppose the original schedule produce this graph:
 - o The order is $3 \rightarrow 1 \rightarrow 2$
- A schedule is equivalent if it has the same behaviour (so we need to draw its conflict graph and check the behaviour

Ottimizzazione Query

Cardinalità

- Per il calcolo dei join normali, moltiplico cardinalità dei due rami e divido per la tabella che ha l'attributo del join come UNICA chiave primaria
 - o Es: join su CodLibro, divido per tabella Libro(CodLibro, Nome, Autore, Prezzo) e non su Pubblicazione(CodLibro, CodTempo, CodRevisionatore) in quanto devo vedere gli elementi distinti
- Per il calcolo dei semi-join, c'è tutto un discorso dietro. La formula base è $\text{CardinalitàOuterQuery} * \text{Fattore di Riduzione}$, dove il fattore di Riduzione è pari a $\frac{\text{ElementiDistintiInnerJoin}}{\text{CardinalitàTabellaCodiceJoin}}$
 - o Per farla veloce, se la cardinalità dell'inner join dovesse risultare maggiore della cardinalità della tabella con chiave primaria il codice del join diventerebbe impossibile stimare il numero di elementi distinti, dunque il fattore di riduzione sarebbe semplicemente circa 1
 - o Dunque, il risultato del semi-join sarebbe circa $\text{CardinalitàOuterQuery}$
- Per la cardinalità a seguito di una GB, prima di eventuali selezioni, si deve prendere il valore di cardinalità più piccolo tra i rami che contengono l'attributo sul quale si esegue la GB; eventuali fattori di riduzione legati a selezioni vanno poi moltiplicati successivamente

Indici

- Si mette un indice se:
 - o La cardinalità della tabella è $\geq 10^3$ e il fattore di riduzione è $< 1/10$ (ENTRAMBE LE CONDIZIONI)
- Il tipo di indice dipende dall'elemento cui effettuiamo la selezione
 - o Range numerico => B-Tree+
 - o Elemento netto (Es: città = 'Torino') => Hash

Anticipo GroupBy

- Se è presente un semijoin, posso anticipare innanzitutto nel ramo dell'outer query, se l'attributo di selezione dipende da quel ramo (se ho select (budget > 2000) e nell'outer non ho il budget non ha senso anticipare lì, però potrei anticipare nell'inner query)
- Se è presente un join, posso anticipare solo se (ENTRAMBE LE CONDIZIONI DEVONO ESSERE VERIFICATE)
 - o Il join avviene sulla stessa chiave della group by
 - o Ha senso logico effettuare il join in quel ramo (se per esempio dopo il join segue una select che in quel ramo specifico non riduce il numero di elementi anticipare lì la GB non migliora il risultato del join e, di conseguenza, peggiora il risultato generale rispetto a lasciarlo a un "livello" superiore

MongoDB

\$addFields

How it works: Adds new fields to documents, computing values using expressions.

Placement: Usually after \$match or \$group to add computed fields.

Example:

```
db.collection.aggregate([
  { $addFields: { totalPrice: { $multiply: ["$price", "$quantity"] } } }
])
```

\$count

How it works: Returns the number of documents.

Placement: Usually after \$match.

Example:

```
db.collection.aggregate([
  { $match: { status: "active" } },
  { $count: "totalActiveUsers" }
])
```

\$group

How it works: Groups documents by identifier and performs aggregation operations.

Placement: Typically used after \$match or other stages to group data based on a field and apply aggregation operations.

Example:

```
db.collection.aggregate([
  {
    $group: {
      _id: "$category", // Group by category field
      totalQuantity: { $sum: "$quantity" }, // Calculate total quantity for each category
      avgPrice: { $avg: "$price" } // Calculate average price for each category
    }
  }
])
```

\$match

How it works: Filters documents based on conditions.

Placement: Usually one of the first stages to reduce processing time.

Example:

```
db.collection.aggregate([
  { $match: { status: "active" } }
])
```

\$project

How it works: Specifies which fields to include or exclude.

Placement: Typically after \$match or \$group.

Example:

```
db.collection.aggregate([
  { $project: { name: 1, age: 1 } }
])
```

\$set

How it works: Adds or modifies fields.

Placement: Same as \$addField.

Example:

```
db.collection.aggregate([  
    { $set: { status: "processed" } }  
])
```